1 Lesson 12-11-26

1.1 Peripheral examples

Let's now look at a few peripheral examples that we might find in actual SoCs.

1.1.1 GPIO

GPIO stands for *General Purpose Input Output* controls or checks the logic state of a set of pins, used to control a variety of devices.

We have a set of **control registers** for a variety of options per pin:

- Direction (for bidirectional GPIOs);
- Edge Detection;
- Interrupt generation;
- Internal pull-ups/pull-downs.

From the **status registers** we can obtain info about *edges detected*. From the **data registers** we can fetch the actual *pin value*.

This can be used to control lamps, piezos, relays, ... on the output side, and buttons, switches, ADCs, ... on the input side. Of course electrical considerations apply (resistances when controlling LEDs or BJTs, ...).

1.1.2 Timers

A **timer** is used to count clock ticks. They use the same processor clock, fed to a counter. Number of bits of the counter and clock frequency determine both the *resolution* of the timer, and the *range*. We can trade resolution for range by using a **prescaler** which dides the processor clock frequency by an arbitrary integer value, not necessarily a power of 2. The terminal Count (**TC**) of the pre-scaler drives the enable of the counter. The number of bits of pre-scaler determines the maximum divisor.

At this point, different types of timers are available:

- **Input capture**: we set an external source (edge or level detected), start the timer. When the external source is detected we read the counter value;
- **Output compare**: we set a count and start the timer. When the count is finished, we drive a line high or pulse it.
- **Function generator**: output compare timers can automatically reset themselves: this means they can toggle a line periodically, building square wave oscillators;
- **Interrupt generator**: often timers raise interrupt lines when they finish counting.

Counters are similar to timers, but they count **events** (not *clock ticks*). Usually events are transitions on an external pin. If a timer and a counter are combined, we can compute both the number of events and their frequency over time.

The peripheral registers are the same as a timer, where we configure the input source.

A **watchdog** timer is a down counter, initialized to some known value. Clock signal as input, it measures time (as any timer does).

When it reaches 0, it generates a non-maskable reset to the system: the software shall reload the initial value before the deadline expires.

This is used to restart a system in case it hangs up due to an error or transient failure. It is useful to recover when no user intervention is possible, so it often is present on SoCs for embedded systems.

1.1.3 PWM modulators

A **PWM** (*Pulse Width Modulation*) modulator allows to send signals "close to analog" without having actual analog outputs (DACs). This is done by quickly alternating a pin between the two voltage levels V_H and V_L, with a high period (duty cycle) proportional to the clock frequency by the analog value we want to output.

In symbols:

$$f = \frac{1}{T}, \quad T = T_H + T_L, \quad \delta = \frac{T_H}{T} \text{ (duty cycle)}$$

The output signal has an average δV_H, which we hope to get close to, with little *ripple* (high frequency noise), when driving an external circuit with some capacitance (which makes it act as a low pass filter).

If δ is slow compared to f, we can use an actual filter to extract the waveform of the average value of the PWM square wave. The average value follows the modulation signal of the duty cycle δ.

How is this implemented?

- A PWM generator consists of a counter, a register and a comparator.
- The counter increments up to a configurable `MAX` value, then resets to 0. A pre-scaler divides the processor clock frequency f_{ck} by an integer divisor.
- The register stores a configurable value $\texttt{reg} \in [0, \texttt{MAX}]$. The output is:

$$\text{PWM}_{out} = \begin{cases} 1 & \text{if } \texttt{cnt} < \texttt{reg} \\ 0 & \text{if } \texttt{cnt} \geq \texttt{reg} \end{cases}$$

- The output duty cycle is:

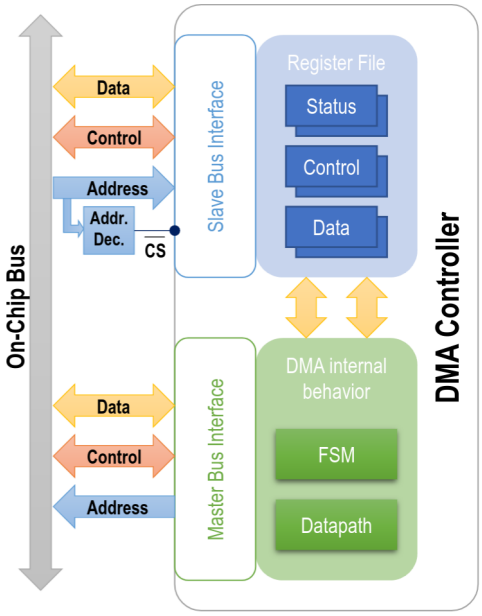
$$\delta = \frac{\texttt{reg}}{\texttt{MAX} + 1}$$

1.2 DMA

Other peripherals we are interested in involve **DMA**. First, we need to distinguish between the three main peripheral access modes in a system:

- **Polling** (*busy waiting*): the CPU continuously checks the peripheral's status until it is ready, wasting CPU cycles;
- **Interrupt**: the peripheral notifies the CPU when it requires attention. The CPU can perform other tasks in the meantime and execute an interrupt handler when notified;
- **DMA** (*Direct Memory Access*): a dedicated controller transfers data directly between memory and peripherals, without requiring the CPU to handle each individual transfer.

DMA is clearly the fastert, but it requires a **DMA controller** to be present on the bus.



As we can see, a DMA Controller has 2 interfaces:

- A **Slave** Bus Interface to access the peripheral registers, the same as a standard peripheral. This is used by the microprocessor core to:
  - Configure the DMA controller
  - Check its status
- A **Master** Bus Interface to generate bus transactions. It is used by the DMA controller internal behavior. It competes with other masters for the On-Chip Bus ownership (meaning we need arbitration on the On-Chip Bus). A DMA often has higher priority than a microprocessor core.

1.2.1 External memory

Similar to DMA (as in, it has 2 interfaces) is an **external memory controller**.

